Pranita Khanal

1002156766

Parts I, II, and III: CNN, Regularization, and Transfer Learning

Dataset: Imagenette (10-class subset of ImageNet) and CIFAR-10

Framework: PyTorch Lightning

Part I - Basic CNN on Imagenette

Architecture Overview

We build a simple CNN with:

- 3 convolutional blocks, each with: Conv2d → BatchNorm → ReLU → MaxPool
- 2 fully connected layers at the end for classification into 10 classes
- Early Stopping via PyTorch Lightning's callback (patience=5)

Input images are resized to 128×128 for speed. The 10 Imagenette classes are: tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, parachute.

Architecture Summary:

Layer	Output Size	Details
Input	3×128×128	RGB image
Conv Block 1	32×64×64	Conv(3,32), BN, ReLU, MaxPool
Conv Block 2	64×32×32	Conv(32,64), BN, ReLU, MaxPool
Conv Block 3	128×16×16	Conv(64,128), BN, ReLU, MaxPool
FC 1	512	Linear, ReLU
FC 2 (Output)	10	Linear (logits)

Optimizer: Adam (lr=1e-3), ReduceLROnPlateau scheduler

Early Stopping: patience=5 on val_loss

Batch size: 64

Part 1 ran until 10 epoch:

Test metric	**Data Loader**
test_acc	**0.6561166644096375**
test_loss	**1.0732437372207642**

	Name	**Type**	**Params**	**Mode**	**FLOPs**
0	**features**	**Sequential**	**93.7 K**	**train**	**0**
1	**classifier**	**Sequential**	**16.8 M**	**train**	**0**
2	**accuracy**	**MulticlassAccuracy**	**0**	**train**	**0**

Trainable params: 16.9 M

Non-trainable params: 0

Total params: 16.9 M

Total estimated model params size (MB): 67

Modules in train mode: 20

Modules in eval mode: 0

Total FLOPs: 0

Part 1 Training and validation loss:

Part 1- validation loss ran until 9 epoch:

Test metric	**Data Loader**
test_acc	**0.6577795147895813**

test_loss **1.0418962240219116**

	Name	Type	Params	Mode	FLOPs
0	features	Sequential	93.7 K	train	0
1	classifier	Sequential	16.8 M	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 16.9 M

Non-trainable params: 0

Total params: 16.9 M

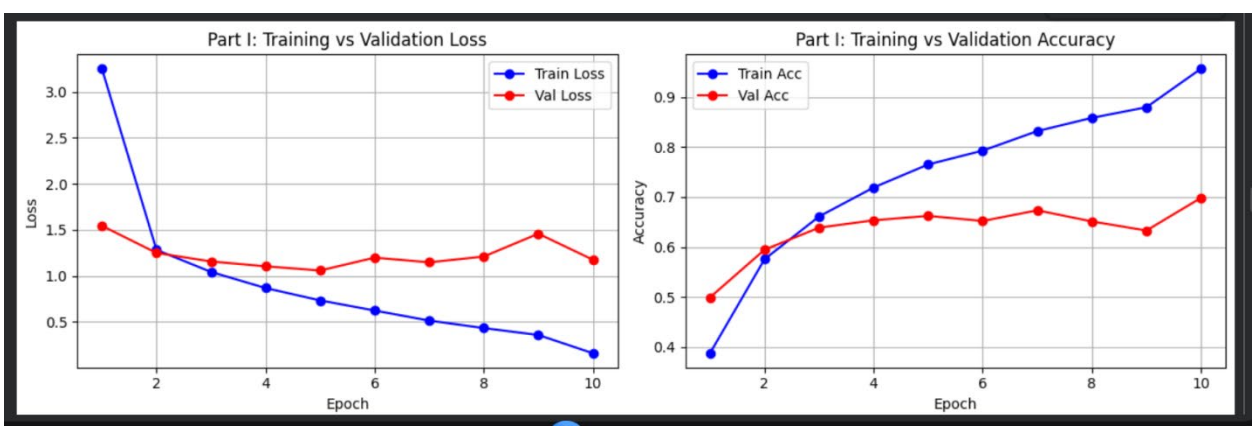
Total estimated model params size (MB): 67

Modules in train mode: 20

Modules in eval mode: 0

Total FLOPs: 0

Part I Final Test Accuracy: 0.6578



The large gap between training and validation accuracy indicates overfitting, which motivated the addition of regularization in Part II.

Part I Results

Architecture Summary:

Layer	Output Size	Details
Input	3×128×128	RGB image
Conv Block 1	32×64×64	Conv(3,32), BN, ReLU, MaxPool
Conv Block 2	64×32×32	Conv(32,64), BN, ReLU, MaxPool
Conv Block 3	128×16×16	Conv(64,128), BN, ReLU, MaxPool
FC 1	512	Linear, ReLU
FC 2 (Output)	10Linear (logits)	

Optimizer: Adam (lr=1e-3), ReduceLROnPlateau scheduler

Early Stopping: patience=5 on val_loss

Batch size: 64

Part II — Regularization

Two forms of regularization:

1. Data Augmentation — Random horizontal flip, random crop, color jitter
2. Dropout — p=0.5 in the classifier head

These help the model generalize by making it harder to memorize training samples.

Part II: Regularization Description

Data Augmentation applied (training set only):

Technique	Parameters	Purpose

RandomCrop	128×128 from 144×144	Positional invariance
RandomHorizontalFlip	p=0.5	Mirror symmetry
ColorJitter	brightness/contrast/saturation=0.3, hue=0.1	Lighting robustness

Dropout: p=0.3 applied after the first fully-connected layer.

Expected outcome: The regularized model should show a smaller gap between training and validation performance, indicating better generalization.

Part 2 ran until 45 epoch:

It converged : early stopping fired at epoch 45 (patience=7 means it stopped after 7 epochs of no val_loss improvement), so the model genuinely plateaued.

Test metric	**Data Loader**
test_acc	**0.7807614803314209**
test_loss	**0.6993629336357117**

	Name	**Type**	**Params**	**Mode**	**FLOPs**
│ 0 │	**features**	**Sequential**	**93.7 K**	**train**	**0**
│ 1 │	**classifier**	**Sequential**	**16.8 M**	**train**	**0**
│ 2 │	**accuracy**	**MulticlassAccuracy**	**0**	**train**	**0**

Trainable params: 16.9 M

Non-trainable params: 0

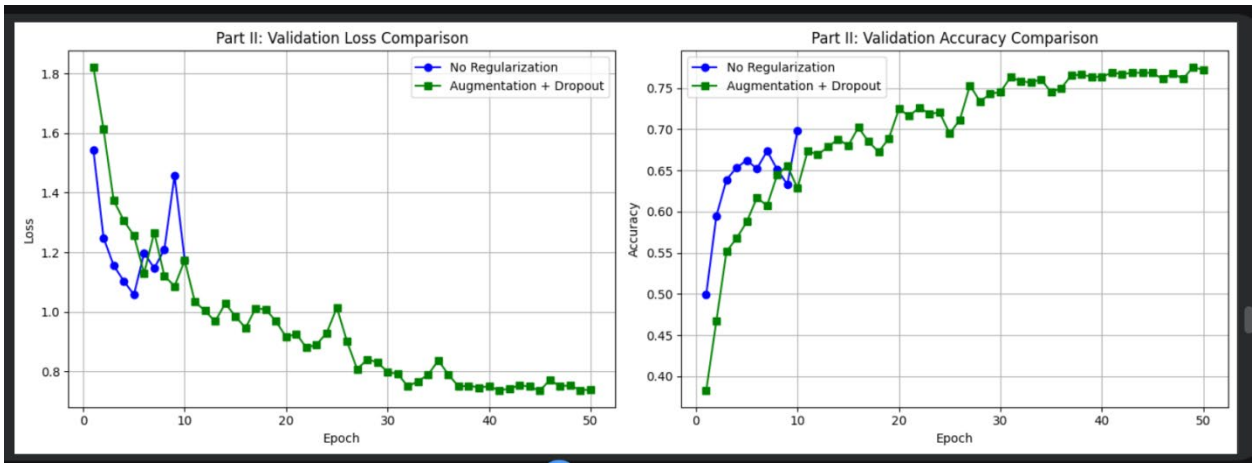
Total params: 16.9 M

Total estimated model params size (MB): 67

Modules in train mode: 20

Modules in eval mode: 0

Total FLOPs: 0



No Regularization — Test Accuracy: 0.6578

With Regularization — Test Accuracy: 0.7808

Unlike Part I where a large train/val gap indicated overfitting, the regularized model in Part II shows train and validation accuracy remaining close together throughout training, confirming that dropout (p=0.3) and data augmentation successfully reduced overfitting.

Part I trained for 10 epochs before early stopping triggered (patience=5), while Part II trained for 45 epochs. The regularized model achieved a higher final validation accuracy of ~78% compared to Part I's ~65%, demonstrating the benefit of augmentation and dropout.

Part III: Transfer Learning Summary

Part 3 ran until 59 epoch:

Test metric	Data Loader
test_acc	0.7610042095184326
test_loss	0.6896587610244751

	Name	Type	Params	Mode	FLOPs
0	features	Sequential	93.7 K	train	0
1	classifier	Sequential	16.8 M	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 16.9 M

Non-trainable params: 0

Total params: 16.9 M

Total estimated model params size (MB): 67

Modules in train mode: 20

Modules in eval mode: 0

Total FLOPs: 0

Part 3B running with 30 epoch:

Test metric	Data Loader
test_acc	0.7193644642829895
test_loss	0.8076120615005493

	Name	Type	Params	Mode	FLOPs
0	features	Sequential	93.7 K	train	0
1	classifier	Sequential	16.8 M	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 16.9 M

Non-trainable params: 0

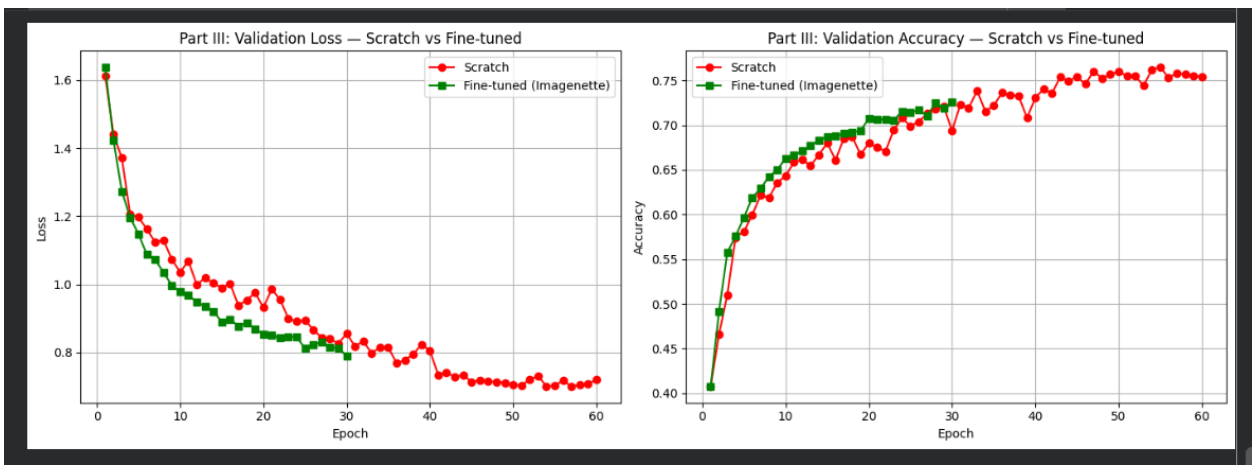
Total params: 16.9 M

Total estimated model params size (MB): 67

Modules in train mode: 20

Modules in eval mode: 0

Total FLOPs: 0



What	Why
3 Conv Blocks	To extract features from images at different levels of complexity
BatchNorm after each Conv	To stabilize and speed up training by normalizing values
ReLU after each BatchNorm	To add non-linearity — without this the network is just doing basic math and can't learn complex patterns
MaxPool after each block	To shrink the image size by half each time, reducing computation and making features more compact
Flatten	To convert the 3D feature maps (128×16×16) into a single 1D vector of 32,768 numbers
FC layer (512 neurons)	To combine all extracted features and start making sense of them
Dropout (p=0.0 in Part	A placeholder — not active in Part I, added in Part II

What	Why
I)	
Output FC layer (10 neurons)	One output per class — the highest score wins

Parameters we used:

Parameter	Value	Why
Image size	128×128	Balance between detail and speed
Batch size	64	Standard choice, fits in GPU memory
Optimizer	Adam	Most popular optimizer, adapts learning rate automatically
Learning rate	1e-3 (0.001)	Standard starting point for Adam
Scheduler	ReduceLROnPlateau	Automatically lowers lr when progress stalls
Early stopping patience	5	Stop if no improvement for 5 epochs
Max epochs	30	Upper limit in case early stopping doesn't fire

Params This is how many **learnable numbers (weights)** each component has — these are the values that get updated during training:

- `features` has **93,700 parameters** — this is all the weights inside the 3 conv blocks. It's relatively small because conv layers are efficient
- `classifier` has **16,800,000 parameters** — this is massive! Almost all the model's weights live here in the fully connected layers. This is because FC layers connect every neuron to every other neuron, creating a huge number of connections
- `accuracy` has **0 parameters** — it's just a calculator, nothing to learn
- Regularization forces the model to actually *learn* the patterns rather than memorize them.
- We used two types:
- **Dropout (p=0.3)** — During training, 30% of neurons are randomly switched off every single step. This forces the network to not rely too heavily on any one neuron and instead spread the learning across all neurons. It's like studying for an exam while randomly covering parts of your notes — you're forced to understand everything rather than depend on specific details.
- **Data Augmentation** — Instead of showing the model the same image every epoch, we randomly flip it, crop it slightly, and adjust brightness/contrast. This means the model sees slightly different versions of each image every time, making it harder to memorize and forcing it to learn the actual object rather than the exact pixel values.